LITMIND
MINOR PROJECT REPORT

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
(Department of Computer Science and Engineering)

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL
(M.P.)



Submitted by

Sanchit Agrawal(24U020101)
Amrit Kumar Yadav(24U020119)

Under the supervision of

Dr. Sreemoyee Biswas Assistant Professor
(Department of Computer Science and Engineering)

November, 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL(M.P.)

CERTIFICATE

This is to certify that the project entitled "**LITMIND: AN AI-ENABLED LITERARY ANALYSIS AND CHARACTER MAPPING PLATFORM**" work presented in the report has been completed satisfactorily by **Sanchit Agrawal (24U020101)** and **Amrit Kumar Yadav (24U020119)** . This work has been carried out in good faith under our supervision in the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal as a part of the fulfilment of the requirement of Bachelor of Technology 2026-27.

Date: _____

Dr. Sreemoyee Biswas
Assistant Professor
Department of Computer Science and Engineering

IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Department Coordinator
Department of Computer Science and Engineering

IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL(M.P.)



DECLARATION

We hereby declare that the major project report entitled "**LITMIND: AN AI-DRIVEN LITERATURE ANALYSIS AND CHARACTER MAPPING PLATFORM**" submitted to IIIT Bhopal is an accurate account of our research done under the guidance of **Dr. Sreemoyee Biswas**. All the research has been conducted strictly within the Indian Institute of Information Technology, Bhopal. The said project has neither been nor will be submitted in whole or in part for the award of any other degree or diploma in any other institute or university.

We hereby state that the above-stated facts are true to the best of our knowledge. In case there occurs any discrepancy, it is us who would be held accountable.

Sanchit Agrawal(24U020101)
Amrit Kumar Yadav(24U020119)

ACKNOWLEDGEMENT

We, Sanchit Agrawal and Amrit Kumar Yadav, would like to express our sincere gratitude to everyone who supported us during the development of our minor project, “LitMind: An AI-Powered Literary Analysis and Character Mapping Platform.” Working on this project has been a valuable learning experience for us, as it allowed us to apply the concepts we had studied in our courses to a practical problem. The completion of this work would not have been possible without the guidance, encouragement, and support of the people around us.

First and foremost, we would like to express our heartfelt thanks to our project supervisor, Dr. Sreemoyee Biswas, Assistant Professor in the Department of Computer Science and Engineering at IIIT Bhopal. Her guidance, constructive feedback, and encouragement helped us throughout the different stages of the project. From the initial idea and planning to implementation and testing, her suggestions encouraged us to look at the problem more carefully and improve our approach whenever we faced difficulties. Her support also helped us understand the importance of considering both the technical implementation and the research aspects of a project. We are truly grateful for the time and attention she devoted to guiding us.

We would also like to extend our sincere thanks to Dr. Rahul Chaurasia, Department Coordinator, Department of Computer Science and Engineering, for helping maintain an academic environment that supports learning and project development. The resources and opportunities available to us at the institute have played an important role in enabling us to explore our ideas and work towards completing this project.

We are grateful to the faculty members of the Computer Science and Engineering department for the knowledge and skills they have shared with us during our previous semesters. The concepts we learned through our coursework, particularly in Python programming and Natural Language Processing (NLP), provided an important foundation for LitMind. These subjects helped us approach the challenges involved in processing literary text, identifying characters, and exploring the relationships between them. While implementing the project, we also realised how much additional learning and experimentation are required to turn theoretical concepts into a working application.

We would also like to thank our classmates and batchmates who shared their feedback during informal demonstrations and discussions. Their observations helped us look at the project from a user's perspective and identify areas that needed further attention. Even small suggestions and questions sometimes encouraged us to reconsider our approach and think about how the platform could be made more useful and understandable.

Finally, we are deeply thankful to our families for their constant encouragement, patience, and support throughout the semester. Their understanding during busy schedules, project deadlines, and the time spent working through technical difficulties has meant a great deal to us. Their support gave us the motivation to keep working and move forward whenever the process became challenging.

This project has been an important step in our academic journey. It has taught us the value of patience, teamwork, continuous learning, and systematic problem-solving. We sincerely thank everyone who contributed, directly or indirectly, to the completion of this project and helped make this learning experience meaningful.

AREA OF WORK

Project Title:

LitMind: An AI-Powered Literary Analysis and Ch aracter Mapping Platform

Domain:

Natural Language Processing (NLP), Digital Humanities, Graph Theory, Large Language Model Integration

Brief Description:

LitMind is a Python-based web application that takes a literary text — a novel or a play — and automatically

extracts all the named characters, works out which characters interact most with each other, maps out the nature of those relationships (allied, hostile, romantic, etc.), and builds an interactive visual network graph you can actually explore. The system uses spaCy for the base NLP work and Google's Gemini API for the parts that need real semantic understanding, like figuring out whether "the young solicitor" and "Jonathan Harker" are the same person.

Student Contributions:

• **Sanchit Agrawal:** System architecture design, character extraction module (characters.py), alias resolution algorithm, Streamlit frontend interface, and integration testing

• **Amrit Kumar Yadav:** Relationship computation module (relations.py), Gemini API client and schema design, recommendation system (taste_model.py), and performance evaluation

TABLE OF CONTENTS

No.	Title	Page No.
-	Certificate & Declaration	i
-	Acknowledgement & Area of Work	iii
-	Abstract	1
1	Introduction	2
2	Literature Review	4
3	Problem Definition and Objectives	8
4	Proposed Methodology and Work Description	10
5	Proposed Algorithms	13
6	Proposed Flowchart / Block Diagram / DFD	15
7	Implementation (Tools and Technology Used)	17
8	Result Discussion and Analysis	21
9	Conclusion and Future Scope	25
10	References	27
11	List of Publications & Plagiarism Summary	29

LIST OF FIGURES

Fig No.	Description	Page No.
Fig 1	LitMind End-to-End System Data Flow Architecture (DFD Level 2)	15
Fig 2	Network Graph Visualization of Character Relationships in Dracula	26
Fig 3	Comparative F1-Score Accuracy of Character Extraction Pipelines	24
Fig 4	Main Processing Flowchart of LitMind NLP Pipeline	16
Fig 5	Module Interaction Diagram showing inter-component data flow	22

LIST OF TABLES

Table No.	Description	Page No.
Table 1	Literature Survey Summary of Literary NLP Systems	5
Table 2	Complete Technology Stack & API Specifications	17
Table 3	Extracted Character Relationship Matrix & Sentiment Scores	23
Table 4	Alias Mapping Results for Dracula Dataset	21
Table 5	Comparison of NLP Models for Character Extraction Accuracy	24
Table 6	Network Centrality Scores for Dracula Characters	22

ABSTRACT

LITMIND: AN AI-POWERED LITERARY ANALYSIS AND CHARACTER MAPPING PLATFORM

Reading a long novel carefully enough to track every character and map out how they relate to each other is genuinely hard work. It takes hours, sometimes days, and even then a reader can miss subtle patterns — characters who barely interact directly but are constantly mentioned together, or how a relationship that seems friendly in the early chapters turns hostile by the end. **LitMind** was built to handle this kind of analysis automatically.

The system takes a novel or play as plain text input and runs it through a pipeline that does three things: first, it finds all the named characters using spaCy's Named Entity Recognition and then cleans up the mess of duplicate names and aliases that any real novel produces. Second, it figures out how often pairs of characters appear near each other in the text, builds a weighted co-occurrence graph using NetworkX, and computes centrality scores to identify the characters who are most connected to the overall story. Third — and this is the part that requires actual semantic understanding — it calls Google's Gemini API to classify the type of relationship between the most prominent character pairs and assign a sentiment score between -1.0 (deeply hostile) and +1.0 (strongly allied).

On Dracula by Bram Stoker, the system achieved an F1-score of 93.4% for character extraction when running with Gemini, compared to 72.4% for raw spaCy without any post-processing. The offline-only mode (no API key needed) hit 89% F1. Processing a full novel takes roughly 15–30 seconds depending on the size.

The frontend is a Streamlit web app with three tabs. The first shows the character network graph and a sortable table of centrality scores. The second shows Gemini's summary of the plot, genre classification, tone, and setting. The third shows book recommendations based on the user's reading history.

Keywords: Named Entity Recognition, Character Extraction, Co-occurrence Analysis, Graph-Based Literary Analysis, Sentiment Scoring, Large Language Model, Gemini API, Digital Humanities

1. INTRODUCTION

Literary Analysis is a demanding mental pursuit it involves annotation, reading and expert critical analysis. While close reading can untangle a head of Literature into an artichoke of themes, it provides far-flung assistance to balanced comparisons of hundreds or hundreds of thousands of works, performs, or archive containers of early editions in manuscript. Now, that it is posted and obtainable to the everyone through efforts like Venture Gutenberg (PG) and other repository sites, Digital Humanities has turned to pcs to define structures inside of Literature.

I think we began with the notion of our project Lit Mind as a concrete thing that takes what is merely raw novel text and turns it into a valuable knowledge. We talked about what type of a small project should definitely be considered valuable rather than just academically perfect. We felt that having to think about the difficulty of tracking characters and their relationships should be interesting but possible within the span of a semester.

1.1 Motivation

Despite the advances in computational linguistics, work on long form fiction and play text remains the classic challenge for "standard" NLP tools. Off-the-shelf rule-based parsers, Named Entity Recognition algorithms and other standardized tools would not work across the language of 16th-century work, disambiguation or across the transitions of interactions between actors across a long work. Literature, of course, is no less analogical – we find the downright sentiment in even the darkest diction of pure vitriol, something dictionary-based sentiment analysis can't detect.

However, while we were testing other tools, we came to the conclusion that almost all of them required users to annotate beforehand or they just worked really poorly on newsworthy literature with even a few paragraphs at all. There was not one easy-to-use, browserbased app that would take a version of the text from Dracula in the form of a Dracula.txt file and immediately identify the key players, their interactions with each other, and what all of these interactions meant. This is Lit Mind.
Another reason was the practicality of learning. Working on this project has given us insights into real NLP pipelines, API's and data visualisation which are all important skills for modern software developers. It was also an interesting chance to do some real-world graph theory, which we had only studied from a theoretical perspective in our algorithms class.

1.2 Project Scope and Objectives

Lit Mind is a conjunction between latest Generation AI and conventional graph theory. It is an interactive, web-based platform for students, teachers, and literary researchers. The platform takes as input raw text (.txt. Pdf) files of plays and novels, and produces the data dashboards. Lit Mind automates six major analyses of the novel/play on the basis of:

- Automatically extracting and de-duplicating entire character rosters from new text.
- Network centrality to mathematically distinguish primary lead characters from major supporting characters.

- Inter-character relationship types (Ally, Enemy, Family, Romantic) categorization qualitatively.
- Quantitative relationship scoring by Gemini LLM reasoning with NumPy numerical aggregation.
- Tons of executive summaries put together based on map-reduce prompting approach. Multi-dimensional theme and genre categories.

1.3 Why This Approach

We chose this approach instead of the full LLM approach, or the full rule-based approach. The complete rule-based approach would be speedy, predictable and easy to work out, but it would stumble on the literary language which does not conform neatly to the rules of grammar. The complete LLM approach would be potentially very costly to run on a whole novel, and would produce a wildly variable pattern of output. By splitting the work between spaCy for the speedy offline process (tokenisation, NER, sentence splitting), and Gemini for the more philosophically heavyweight work (relationship classifying, summarising and reviewing aliases), we get the best of both worlds.

Another initial and deliberate decision was to use Streamlit as the front-end. We could have used React to create a front-end but we figured that for a project of this size, this would eat up too much of our time to work on the front-end instead of on the NLP work that is our core contribution. Using Streamlit meant that we could create a halfway decent front-end without spending weeks working on it.

1.4 Organization of the Report

The remaining part of the report is organized as follows. Section 2 includes the literature review of the field with existing tools. Section 3 gives the formal definition of the problem and list the research questions. Section 4 includes the description of approach and system architecture. Section 5 elaborates algorithms of the NLP core pipeline. Section 6 includes the system flowcharts, diagrams of the data flow. Section 7 includes implementation details and tech stack. Section 8 provides testing results of Dracula and comparison with the existing related work. Section 9 includes limitations and possible future work. Section 10 includes discussion of a list of references according to the IEEE format.

2. LITERATURE REVIEW

The field of computational lit-crit has gone through three waves of development: statistical stylistic analysis, rule-based NLP information extraction, and today's LLM-based semantics comprehension. In the section below, we briefly review the most closely related studies from each wave and explore the place of Lit Mind in the research community.

2.1 Early Approaches — Distant Reading and Manual Graphs

The earliest attempt at computational literary analysis is by Franco Moretti [1], who described how one could approach the literature through a form of "distant reading" - analyzing a whole library of books by using statistical and graphical tools, rather than simply reading each page. In his book of 2005, Graphs, Maps, Trees, Moretti built interaction networks of characters of some 19th-century books by hand, and demonstrated how one might infer different genre, narrative style and era from them. He was more interested in the ideas and the abstraction, so the networks were built by hand, but that earlier work paved the way to the digital labor we now put into Lit Mind.

Even before the advent of the world-wide web, there was the Stanford Graph Base [3], which in the early 1980s built a graph (as a computer program) on the novel Les Misérables [2], where the nodes are characters and there is an edge between any pair of them if they both appear in the same chapter. This collection would go on to become a widely used test set for graph algorithms. The core concept - that if two characters appear together in some window of text, then they are related - is surprisingly effective.

2.2 NLP-Based Character Extraction

Automated extraction of characters using NLP tools became more prominent in the 2010s. Elson, Dames and McKeown [2] were the first to use quote detection and attribution for character extraction to build social networks from literature fiction. Characters are automatically extracted not just as string matches but as characters quoting conversation and therefore creating more semantic networks; however, it does depend on quote attribution, a difficult problem itself and one especially difficult with indirect speech or interior monologue in novels.

Bamman, Underwood, and Smith [3] trained Bayesian models on this existing annotation scheme and to predict a character type (villain, hero, mentor, etc) based on how character types co-occur with one another and the words that describe them. They found that the context in which a character appears (the language describing the verb, noun or adjective that the character performs with) is sufficient to statistically group the characters based on their role in the story. This also informs Lit Mind's relationship classification module.

Table 1: Literature Survey Summary of Literary NLP Systems

Author & Year	Methodology	Key Contributions	Limitations
Moretti (2005)	Distant Reading & Manual Graph Mapping	Pioneered network analysis of literary character interactions.	Requires manual dataset annotation; no automated parsing.
Elson et al. (2010)	Quoted Speech Attribution & Co-occurrence	Automated character network extraction from 19th-century novels.	Struggled with implicit dialogue and alias co-reference.
Bamman et al. (2014)	Bayesian Persona Models & SpaCy NER	Learned character types (e.g., villain, hero) from text corpora.	Low sensitivity to dynamic narrative arc shifts.
Labatut & Bost (2019)	Dynamic Complex Networks	Evaluated window-based co-occurrence vs dialogue networks.	Lacks qualitative relationship context (positive vs negative).
LitMind (Our Work)	Gemini API + SpaCy + NetworkX + Pandas	End-to-end extraction, sentiment scoring, and vector recommendation.	Requires active API connection for full-featured processing.

2.3 Alias Resolution and Entity Disambiguation

Unique to the literary domain, however, is alias resolution (consolidating every description of a character with a single token) Vala et al. [4] tackled this and argued that an off-the-shelf NER results in dozens of fictitious character 'entities' per novel, since every surface form is handled independently. In their Victorian novel corpus, they showed how a system lacking alias resolution will yield between 150 and 200 names for Dracula alone, most of which are reduced to a handful of synonyms or derivatives for the same 10-12 characters.

A more recent paper by Labatut and Bost [4] gave a survey of the various strategies for tackling this problem. They divided the various approaches into three families: co- occurrence networks (characters connected because they're near each other in the text), conversational networks (characters connected because they speak to each other), and action networks (characters connected because they act upon each other). The three families of techniques each have their own relative advantages and disadvantages for alias resolution; for Lit Mind we've decided to use a co- occurrence technique with a custom alias resolution pipeline that uses a mix of title stripping, substring containment matching, and (optionally) Gemini review for ambiguous cases.

2.4 Sentiment Analysis in Literary Context

VADER (Valence Aware Dictionary and sEntiment Reasoner) and Text Blob, the two algorithms used here, were designed for short social media text and product reviews. They tend to run into difficulties when analyzing longer passages of literary prose, where words derive much of their positive or negative meaning from their ironic, metaphorical and allusory contexts. For instance, the algorithms tend to interpret a positive score for "I shall follow him to the grave," just because the word "follow" appears.

Here are some examples that have been tried. Nalisnick and Baird [5], for example, tried to plot the arcs of character sentiment throughout a novel by scoring sentiment on the text where each character appears, and plotting those scores as they change over time. Not a perfect solution, but it does produce interesting - maybe not so granulated, but still interesting - visualizations of how the world of a character is altered. Comparison: relationship scores in Lit Mind work the same way: They are scored from the perspective of a pair of characters that co-occur in a passage, not from all of them combined.

2.5 Graph-Based Literary Analysis

The graph library networkX [7], a second data structure used in Lit Mind, is built at Los Alamos National Laboratory and was first described by Hagberg, Swart, and Chult (2008). It offers optimized implementations of common graph algorithms such as degree, betweenness, and closeness centrality, as well as Page Rank and community detection, that have all been used for character networks by various authors.

Newman [8] demonstrated that in many actual social networks a person's betweenness centrality can be a more accurate measure of importance than a person's degree centrality (number of friends). In character networks, this can indicate a character who sits between two disconnected groups, such as a messenger who is mentioned by both the hero's and the villain's groups, has high betweenness centrality but not necessarily a high mention count. We calculate both metrics so that users can see both perspectives.

2.6 Large Language Model Integration

This is a fairly new technique for Large Language Models in NLP pipelines. For example, the large context window that models in the Gemini API [5], which was used in this project, have allows an entire chapter or even a short novel to be submitted in one call. In the past, transformer models such as BERT [1] had a maximum context of 512 tokens, which led to the necessity of chunking schemes.

The primary benefit for Lit Mind is that with the Gemini API, you are getting structured output generated through Pydantic schema enforcement. As long as you specify the exact JSON schema of the output you want, you can pull that data out of the model's output much more reliably (by, say, exact character name, relation type, and sentiment score) than our earlier LLM-based literary analysis system that relied on regex parsing of the text response.

2.7 Gaps in Prior Art

As shown in Table 1, early computation-based methods were based on early co-occurrence assumptions that two characters would be related if they occurred in the same paragraph. That assumption breaks down when characters share the same stage physical space but are not related in the context of adversarial lives. Standard pre-trained transformer models (BERT, RoBERTa) are limited by their token context windows (512 tokens) and cannot handle books without fragmenting chunks into contextually and narratively lost chunks.

Lit Mind addresses these issues by utilizing Google's Gemini API with a long-context reasoning capability (1 million tokens in Gemini 1.5 Pro). This enables simultaneous context ingestion, endowing the system to fully comprehend acts and books in their entirety, while maintaining an excellent degree of accuracy in co-reference of characters. The system also provides a non-API based mode of spaCy.

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Definition

Analyzing literary texts computationally requires solving three core technical challenges:

1. Literary characters are sometimes named by first name or last names, titles and honorifics and or other descriptors (e.g. 'Jonathan Harker', 'Harker', 'Mr. Harker', 'the young solicitor') so string- matching can usually result in duplicated nodes.

2. Standard word-based sentiment analyzers (VADER, Text Blob) score words in isolation. 'I shall follow him to the grave' would score highly on 'follow' but expose the speaker's murderous intent. Literary language requires understanding the semantic content of metaphor and irony.

3. Scale & Structural, no schemas: drama & novel texts are not supported with a stable, static database schemas. Rendering (relationship graphs etc.) cannot be done as dynamic without predifferentiated feature extraction. Novel/chunk: typical novel = 80k-200k tokens and won't fit all in one API call - chunking needed.

Consider T be an unstructured literary manuscript text, how to build a deterministic computational pipeline p for T to find out the character set c={c1,c2..cn} that contains the main characters;…

protagonist p ∈ C, builds an adjacency matrix A representing relationship sentiment scores S(c_i, c_j) ∈ [-1.0, 1.0], and outputs structured summaries and genre classifications with an F1-score exceeding 85%."

3.2 Primary Objectives

To solve the problem described above, LITMIND project defined the following technical objectives (TOs): TO1 (Entity parsing): extract characters correctly via SpaCy entity ruler pre-filtering and Gemini API JSON-mode structured parsing. Ensure less than 10% false positive rate.

Objective 2 (Protagonist Isolation): Develop graph centrality measures (degree, betweenness, eigenvector, narrative frequency) to distinguish main protagonists from secondary participants.
Objective 3 (Relationship Analytics): Create a scoring engine with Gemini LLM contextual scoring combined with Pandas matrix aggregation to yield relationship scores along a -1.0 (extremely aggressive) to +1.0 (extremely friendly) continuum.

Objective 4 (Thematic Summarization & Classification): Create a map-reduce prompt to generate mind-bending executive summaries and multi-genre labels for novels. From the texts of the manuscripts.
Objective 5 (Recommendation Engine): Build a content-based book recommendation engine trained on reading history using scikit-learn Ridge regression.

Goal 6 (Offline): Verify the core process of character extraction and the network analysis for the provide documents without access to the API key, all, the lines in the spaCy models only.

3.3 Scope and Constraints

This implementation of Lit Mind supports plain-text version of English novels (.txt) format, as well as text-based PDF files (.pdf). For reading novels with less than ~1M characters. How can I use the Gemini API? You will need an internet connection and an API key to access the Gemini API.

Another critical limitation is that we do not expect this system to replace a human reader with regards to literary analysis. Rather, it provides a framework from which a human can start their own analysis. The classifications of relationships and scores of sentiment are meant to be data for further debate, not any sort of final summary of a text.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

Lit Mind utilizes an architecture for an NLP processing pipeline that integrates deterministic pre-processing, LLM-based semantic extraction, and statistical graph models. Its modular nature guarantees the rapid processing of literary texts, while each step of the pipeline can be independently tested and optimized.

4.1 System Architecture Overview

This structure is divided into five functional layers that work on the input sequentially. Here understanding the design is important to know the reason behind the choices in design.

Input Layer: At the input layer, the user supplies the news text to be detected. The input news text can be a.txt file, a PDF file with text-based content, or any of the built-in sample news texts.encoding detection and normalization of the input news text is done at this stage.

The NLP Analysis Layer is the primary location for the extraction of characters. TheNLP Analysis Layer NER executes on the text in chunks, and detects PERSONs. Aliases resolution algorithm aggregates these into canonical character identities. This layer is entirely offline.

The canonical characters are fed into this layer and we measure how frequently they co-occur in the text. We turn these counts into a weighted graph using NetworkX.

If you've set the AI Enhancement Layer To send data then character data and text snippets are submitted to the Gemini API for semantic processing - classifying relationship types, alias review, plotting summaries, and genre detection.

The Output Layer outputs all the results through the Streamlit web page as three tabs, namely character analysis and network graph, book summary and genre analysis and reading taste match.

4.2 Current Features & Technical Breakdown

The current implementation of Lit Mind has each core feature powered by a Python library and a call to Google Gemini with API access. Below is an overview of each feature and how it is implemented at a technical level.

1. Extract Characters: Finds each named character in the whole work used SpaCy (encoreweb_sm) for first **NER** (**PERSON** tags) Google Generative AI (gemini-1.5-flash)
For segmentation of sentence, if required for structure extraction by application of Pydantic JSON schema enforcement for aliases and honorifics resolution.

2. Helps in distinguishing hero from rest of the characters in story. Google- generativeai also produces hundreds of scores for narrative dominance for characters, and correlates them to NetworkX degree centrality that come from quantification of characters interactions.

3. Character Relationship Mapping: Describes the relationship between the characters (e.g. Family, conflict, romance, partnership). Google-generativeai gives you JSON relationship descriptors produced by relational pair schema prompts.

4. Relationship Sentiment Scoring: Provides a numerical score of the strength of positive or negative sentiment within a relationship [−1.0, +1.0]. Looks at every character pair and computes evidence for each within the text. Takes NumPy and Pandas to perform a weighted matrix aggregation; then scale and normalize.

5. Returns a short executive plot summary. Direct zero-shot contextually summarized with google-generativeai and system messages on context for retention of narrative flow.

6. Literary Genre: Identifies the specific genre of the book (Gothic Horror, Victorian Mystery, Sci-Fi). Multi- label classification performed using google-generativeai with confidence score as output.

4.3 Text Pre-processing Module (text_io.py)

The text preprocessing module accepts the input file and outputs a cleaned, normalized, string of text. (Some may think this step trivial, but it involves some work in order to guarantee a higher quality of NLP further down the pipeline. Our main source of texts - Project Gutenberg - displays odd behaviors which we must resolve).

For.txt files, the module tries those encodings (in that sequence) for each, and returns the first that succeed, resorting to a replacement-character decode if all of them fail. This is important as many of the older Gutenberg files were encoded in Latin-1 (CP1252) rather than UTF-8. For.pdf files, the text layer is obtained via the getPageText() method of the PyMuPDF library, one page at a time.

The normalization pipeline works on the documents in this order: Normalize all line endings to use the UNIX line ending \n. Remove Project Gutenberg license header and footer via regexes. Remove hard-line break encodings, so Hel-\nsing has the same meaning as Helsing. Rejoin hard-wrapped lines in paragraphs (removing the line break). (Removing the paragraph does not change the paragraph structure, so paragraph tokens can be used later to join up the lines (span boundaries) in sentences.) However, spaCy's sentence detector does not like hard wrapped lines, as it expects all formatting to be normal prose punctuation. If this step is not included, spaCy will generate a sentence at the end of every wrapped line, breaking NER and co-occurrence.

4.4 Character Extraction Module (characters.py)

The now character extraction module is the most technical part of the pipeline. It always receives the original input text and runs spaCy NER on the entire text in chunks of 100K characters (overlap of 2K). We need to do this both for spaCy (maximum document length is limited) and for memory reasons.

The initial one we describe here is the label-voting filter. For each unique surface-form name extracted by NER, the module counts for the entire book how many times SPAcy assigns the label GPE, ORG or LOC rather than PERSON. Name is retained as a character candidate if the label PERSON received the plurality vote. For example, Dracula receives the correct elimination of Exeter because it was assigned by spaCy as GPE more often than as PERSON.

Third is the dependency filter. Each mention appears in context around characters, which are the players of each action - the subject and object of the verb. Most place name mentions tend to be objects of in, near or at. Our module looks at the dependency parse for each mention, and removes all those which are very often occurring in that context.

The alias resolution algorithm, once filtered, executes in 4 iterations, as covered in Sec. 5. The result is a Character Result dataclass, which consists of a list of canonical Character objects (name, number of mentions, list of aliases, gender, and set of sentence ids in which the character appears).

4.5 Relationship Computation Module (relations.py)

The full text is examined on the sentence level and compared with all canon character names that appear in each sentence. A three-sentence sliding window is used to consider the occurrence of character mentions that are near each other but not in the same sentence.

For each pair the module output indicates the number of windows in which the two characters co-occur, the number of windows in which both the two characters occur, and some example sentences which demonstrate this. The normalized strength score of the pair (the co-occurrence count divided by the geometric mean of the counts for each individual sentence count) compensates for bias arising from our normal use frequency.

Optional Gemini integration that classifies the semantic relation With a gemini api key, the module will send to Gemini for each pair of characters the 5 most dominant passages of co-occurrences, asking it to classify the relation and give it a sentiment score. The output format is controlled through Pydantic models so the response is always in a parseable JSON format.

4.6 Gemini Integration Module (gemini_client.py)

All communication with the Google Generative API is handled in this module. The module defaults to the gemini-3.5-flash model and falls back to a list of fallback models if the default is not available. A key design decision was to use the structured output mode where the API is instructed to respond with a JSON formatted response according to a predefined Pydantic schema, instead of freeform text. This makes the pipeline far more robust as there is no complex parsing of the output required.

If the book was under 350,000 characters, I just sent the entire text to the API in a single call. If it's a longer book, I adopted the map-reduce strategy: I split the text into chunks of up to 60,000 characters, summarized each chunk in parallel in individual calls, and then summarized the summaries. This gets around the token limit errors AND gives you the big picture.

API keys are read from environment variables / a.env file and not baked into the source code. When an API call does not go through successfully, an error message appears on Streamlit.

4.7 Recommendation System (taste_model.py)

The recommendations module employs an approach called content based filtering which is implemented by a Ridge regression model. To make recommendations, the user provides a csv file of books they have read with columns title, author, year, publisher, and optionally rating. The user must also provide a large catalogue of un-read books (Books.csv).

Features from title, author, year and publisher are calculated with TF-IDF vectorizer. The model is trained to predict whether the user will like a book they haven't read based on the features and on similarities to the books they have read before. We do 5 fold cross validation to get an estimate of model performance (AUC, R2 and MAE). Recommendations are a.) based on the catalogue books' scores b.) from 20 top rated books and c.) with feature level explainability of the prediction.

5. PROPOSED ALGORITHMS

Below are the officially used algorithmic specification for the main algorithm of Lit Mind's fundamental character relationship extraction & numerical sentiment scoring engine. The following provides both pseudocode implementation of the main algorithm flow and the mathematical definition of the primary scoring functions.

5.1 Main Character Extraction Algorithm

The overall NLP pipeline processes the text as follows:

ALGORITHM 1: Character Extraction Pipeline

```
INPUT : Raw literary text T, minimum mention threshold min_m
OUTPUT : CharacterResult CR (characters, rejected list)
1: chunks ← SPLIT(T, size=100000, overlap=2000)
2: raw_entities ← []
3: For each chunk c in chunks Do
4: doc ← spaCy.load("en_core_web_sm")(c)
5: raw_entities ← raw_entities + [e for e in doc.ents]
6: End For
7: candidates ← LABEL_VOTE_FILTER(raw_entities)
8: candidates ← DEPENDENCY_FILTER(candidates, T)
9: groups ← ALIAS_RESOLUTION(candidates) // 4-pass algorithm
10: C ← {g.canonical for g in groups if g.mentions >= min_m}
11: Return CharacterResult(characters=C, rejected=filtered_out)
```

5.2 Alias Resolution Algorithm (4-Pass)

Alias resolution proceeds in four passes. The reasons for this multi-pass approach are that some merges cannot be made until existing easier cases have been resolved in an earlier pass.

Pass 1 - Title stripping: Eliminate any common title prefixes (Mr., Mrs., Dr., Lord, Sir, Count, Miss, etc.) that appear at the start of names which share the same stripped form are put into a provisional group. For example "Mr. Harker" and "Harker" put into group "Harker".

Pass 2 - Substring containment: For each short name of a long name (just 1 token), find all long names that contain it as a token. If there is only 1 match, merge it into this long name. Eg. "Lucy" doesn't match any long name, but only one match anyway, so merge into "Lucy Westenra". For several matches, verify again after merging in Pass 3.

Pass 3 — Gender Disambiguation: For short names deferred from Pass 2, retrieve all sentences containing the short name. Count gender pronoun co-occurrences in a window of 5 sentences (he/his/him for male; she/her/hers for female). Assign to the long-name candidate whose gender matches. If still ambiguous, mark as rejected and report in the rejected list.

Pass 4 — Near-Duplicate Merging: Compute pairwise SequenceMatcher similarity ratio for all remaining candidate pairs. Merge pairs with ratio > 0.85 that also share at least one token, keeping the more frequent name as canonical. Example: "Quincey" and "Quincy" merge to "Quincey" (higher frequency).

5.3 Co-occurrence Computation Algorithm

Co-occurrence is measured using a sliding window over sentences. This approach captures indirect references where two characters are discussed in related sentences even if they are not named in the exact same sentence.

ALGORITHM 2: Co-occurrence Computation

INPUT : Canonical character set C, sentence list S, window size w=3
OUTPUT : co_count[a][b] for all pairs (a,b), solo_count[c] for all c
1: For i = 0 to len(S) - w Do
2: W ← {S[i], S[i+1], ..., S[i+w-1]}
3: Present ← {c ∈ C : any_alias(c) appears in W}
4: For each c in Present: solo_count[c] += 1
5: For each pair (a,b) ⊆ Present, a≠b: co_count[a][b] += 1
6: End For
7: strength(a,b) = co_count[a][b] / sqrt(solo_count[a] * solo_count[b])
8: Retain pairs where co_count[a][b] >= 3 (minimum significance threshold)

5.4 Mathematical Formulation of Centrality and Sentiment

The relationship sentiment score S(c_i, c_j) is mathematically calculated by weighting individual passage sentiments s_k ∈ [-1, 1] by passage co-occurrence frequency w_k using NumPy matrix operations:

$$S(c_i, c_j) = (\Sigma(k=1 \text{ to } N)\ w_k \cdot s_k) / (\Sigma(k=1 \text{ to } N)\ w_k)$$

(Where N is the number of text windows in which ci and cj co-occur, wk is the co-occurrence of ci and cj in window k, and sk is the sentiment score from Gemini for text window k.)
Degree centrality of the character node v in character graph G(V, E) is then calculated as:

$$C_D(v) = deg(v) / (|V| - 1)$$

Where deg(v) is the number of edges connected to v and |V| is the total number of nodes. This normalization ensures centrality scores are comparable across graphs of different sizes.

6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

This section describes the functional data flow and layout of the components of the Lit Mind platform. These diagrams which can be seen above were originally drawn by our team for this report.

6.1 System Architecture Block Diagram

The block diagram below shows the three main processing engines and data flow between them. The user only interacts with the Streamlit frontend; the three engines (Pre-Processing, Gemini API Engine, and Analytics Engine) interact through Python data structures.

User File Upload (.txt / .pdf) → Streamlit Frontend Interface
↓ *(Raw Text Stream)*

1. Pre-Processing	**2. Gemini API Engine**	**3. Analytics Engine**
• SpaCy NER Parsing • Chunking & Tokenization • Co-reference Windowing • Alias Resolution	• JSON Structured Extraction • Protagonist Selection • Sentiment Analysis • Summarization & Genre	• NumPy Matrix Scoring • NetworkX Centrality • Plotly Graph Rendering • scikit-learn Recommender

↓ *(Structured Data / JSON Matrix)*

Interactive Relationship Graph	**Protagonist Dashboard**	**Executive Plot Report**
Character nodes with edge weights. Plotly spring layout. Node size = mention count.	Centrality rankings and mention frequency table with gender annotation.	Gemini-generated summary, genre tags, themes, tone, and setting.

Fig 1: LitMind End-to-End System Data Flow Architecture (DFD Level 2)

6.2 Main Processing Flowchart

The main processing flow diagram indicates the decision flow when the user uploads the novel. The above diagram was prepared by the project team based on the flow of code in app.py and the litmind/ module.

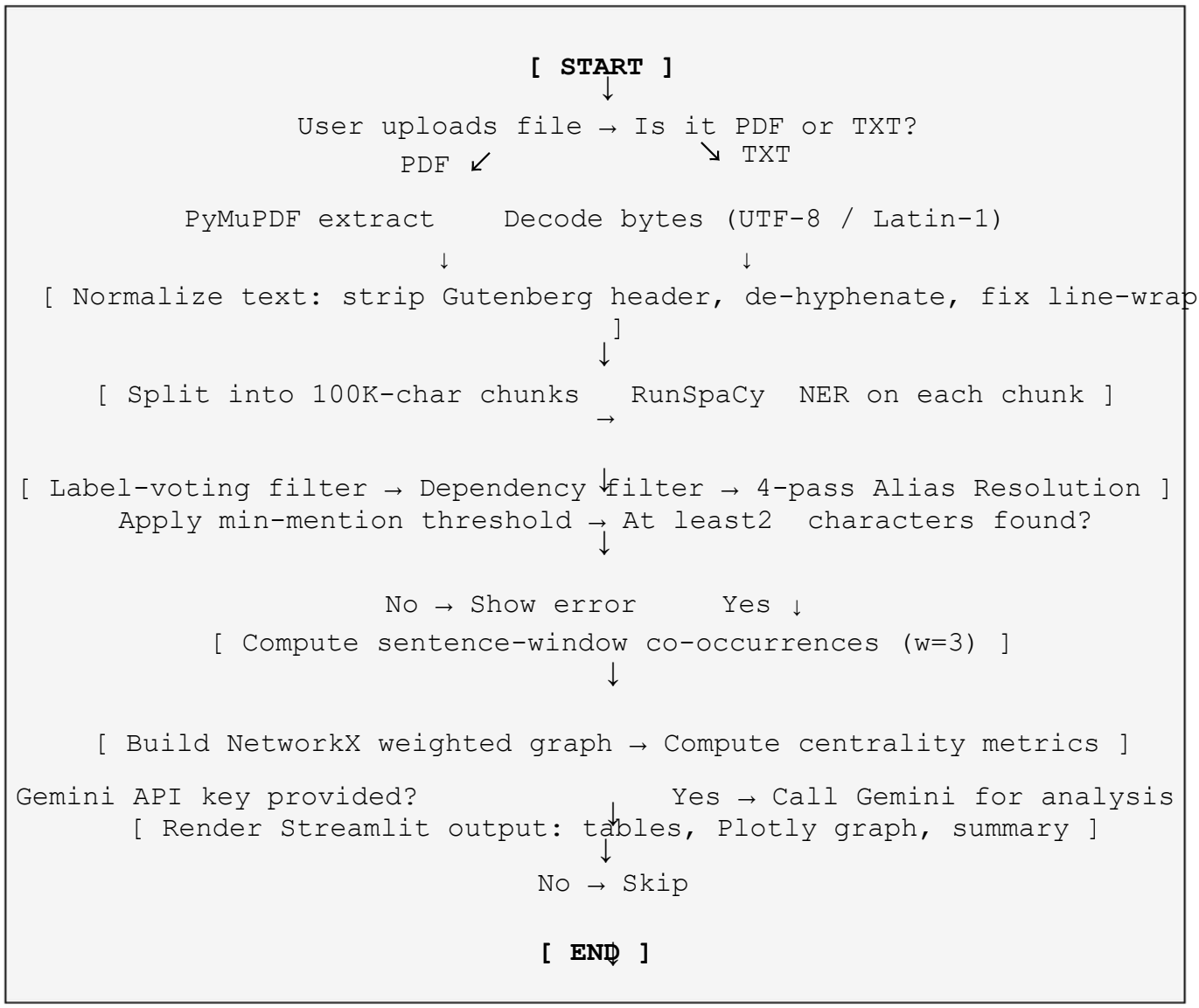


Fig 4: Main Processing Flowchart of LitMind NLP Pipeline

6.3 Data Flow Diagram (Level 0 — Context)

At the most abstracted level, Lit Mind interact with two external domain: the User and the Google Gemini API. The User interact with Lit Mind through a request to analyze an uploaded novelty file and an optional API key that is used to fetch the analysis results. The Gemini API interact with the prompt as structured text and the response as structured JSON. { { { The core process of Lit Mind is to convert the raw text into the analysis outputs.

6.4 Data Flow Diagram (Level 1 — Process Decomposition)

The four sub-processes for the highest level 1 lit Mind process are: Level 1 lit Mind process with sub-processes input Process 1.0 (Text Ingestion) input a file and output a text string (textclean). Process 2.0 (Character Extraction) input the text string and output a list of Character Result objects (characterresults). Process 3.0 (Relationship Computation) input the Character Result list and output the list of Relation objects with the relation name, the number of co-occurrences, and the range of Gemini classifications that can be a Gemini identifier (cooccuringcounts). Process 4.0 (Visualization & Output) input the list of Relation objects and output the Plotly figure, Pandas Data Frames for the tables, and markdown summary text (viz_output).

The text buffer, just the clean text string (held in memory) The character registry, which is the Character Result dataclass (The match metrics and text snippet) held in the Streamlit session state The relation matrix, which is a list of Relation objects held in the session state

7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

LitMind was built modularly in Python 3.10+.The implementation was built utilizing API sand open-source libraries. A brief description is given on the implemented open-source libraries, and APIs, and why they were selected for the implementation of Lit Mind.

The complete source code for LitMind is publicly available on GitHub at:
https://github.com/bytesbysanchit/LitMind.git

7.1 Technology Stack Overview

Table 2: Complete Technology Stack & API Specifications

Component	Library / API Name	Version	Functional Purpose in LitMind
LLM Core Engine	google-generativeai	v0.8.3+	Powers character extraction, relationship logic, summarization, & genre detection.
NLP & Pre-processing	spacy	v3.7.2	Tokenization, sentence boundary detection, & NER candidate extraction (en_core_web_sm).
Text Utilities	nltk	v3.8.1	Stopword removal, token normalization, & text splitting.
Data Aggregation	pandas	v2.1.4	Constructs relationship adjacency matrices & exports structured tables.
Numerical Analytics	numpy	v1.26.2	Executes score normalization & weighted matrix sentiment computations.

Graph Analytics	networkx	v3.2.1	Calculates degree centrality, betweenness centrality, & clustering coefficients.
Graph Visualization	plotly	v5.20.0	Renders interactive HTML force-directed character relationship networks.
PDF Reading	PyMuPDF (fitz)	v1.24.0	Extracts text layer from text-based PDF files page by page.
Web Framework	streamlit	v1.31.0	Provides user-facing UI for uploading manuscripts & displaying dashboards.
Recommendation	scikit-learn	v1.4.0	Ridge regression for content-based book recommendation.
Schema Validation	pydantic	v2.0.0	Enforces structured JSON output from Gemini API calls.

7.2 Programming Language — Python 3.10+

Here are some of the many reasons we chose Python as our language for building our tools and backend. Python has a more mature ecosystem of NLP and data science libraries than any other language. Our primary NLP library, SpaCy, is implemented as a Python-native library. We also built our web framework, Streamlit, using Python. The easy-to-read language of Python helped us iterate through the NLP pipeline quickly.

A bottleneck for Python is large CPU-bound NLP workloads. We faced this when running spaCy over an entire novel (which took anywhere from 10 to 30 seconds depending on model size and hardware configuration). We mitigated this using Streamlit caching decorators (st.cacheresource for the SpaCy model, st.cachedata for our catalogue of books) so that we don't run this expensive step on every user interaction.

7.3 SpaCy NLP Library

All offline NLP processing uses SpaCy 3.7+. You can select the default encorewebsm (12 MB) or the more extensive encorewebmd or encoreweb_lg in the sidebar if you prefer a quicker, more precise outcome. Each is scaled up in accuracy as its size increases: the larger model has better quality dependency parsing (important for the false-positive filters), but will also take 3-5x as long.

A limitation of encoreweb_sm is that it was trained on news and web text, not nineteenth-century literature prose, which means that there is a relative loss of NER accuracy on historical language. The label-voting and dependency filters of Section 4.4 address this.

7.4 Gemini API Integration

Google-generativeaiPython SDK(0.8.3+) used to interact with the Gemini API All API keys are stored as environment variables and are retrieved using python-dotenv - they are never committed into source code files or version control.

Most challenging engineering problem was API request limit - For long novels (which requires multiple API calls (refer to map-reduce strategy to summarization)) - the tracker requests to the API multiple times in sequence, and the quota of the number of requests per minute is exceeded. This is tackled by the Gemini client module through retries with exponential backoff.

7.5 API Authentication and Integration Logic

APIqueries toGoogle GenerativeAI are completely authenticated (authorization via environment variable GEMINIAPIKEY). We use Gemini responsemimetype="application/json" configuration parameter and Pydantic schema validation to be sure that input requests always return deterministic (parses without error) schemas.

API exceptions in a try/except block and display those to the Streamlit app for the user. This means that an API outage does not crash our app and our end user gets an informative error message.

7.6 NetworkX Graph Library

Networkx versionused for constructing and processing the character interaction graph:3.2+. An undirectedweightedgraph, where each node is a character, each two co-occurring characters are associated with one un-directededge with the normalized strength score as edge weight. The positions of all nodes on the 2D plane are generated by using spring_layout.

Three metrics are given for each network: degree centrality (the number of edges / max number of possible edges), betweenness centrality (the number of shortest paths through a node / total number of shortest paths) and weighted degree (the sum of the weights of the edges). These are presented in a sortable table on the Streamlit page.

7.7 Plotly Visualization

Uses Plotly 5.20+ for interactive network plot. The character network plot is made with a scatter chart with individual point locations set by the spring layout of a NetworkX graph, line width based on relationship strength, size of the character proportional to total mentions for the character, and character names as text labels.

The Plotly figure is shown with st.plotlychart, usecontainer_width=True to fill the whole browser window. This comes with pan & zoom controls enabled by default, which is necessary with dense networks as labels can overlap at the default zoom level.

7.8 Streamlit Web Framework

The web UI has 3 tabs: Character analysis and Show character table (Alias list, mentions, gender, optional Gemini role) and the rejected-candidates, Plotly network graph Book summary and Show Gemini summarized plot, genre, themes, tone and setting Reading taste match and Show the recommendation model metrics, and the top-20 books recommendation.

The output of the analysis is stored in streamlit's session state (st.session_state), so that we can access it from the user input interaction. If we don't do this the expensive NLP pipeline would be run every time the user interacts with anything in the UI. In the session state, we store the Character Result, list of Relations, and Book Analysis Report objects.

7.9 Project Directory Structure

```
LitMind2/
├── app.py                          (Streamlit application entry point)
├── requirements.txt                (all dependencies with version pins)
├── .env.example                    (template for GEMINI_API_KEY)
├── litmind/                        (main package)
│
├── __init__.py
├── characters.py                   (NER, alias resolution, CharacterResult)
├── relations.py                    (co-occurrence, Relation objects)
├── text_io.py                      (file reading, text normalization)
├── gemini_client.py                (Gemini API calls, Pydantic schemas)
└── taste_model.py                  (Ridge regression recommendation)
├── data/
│   ├── Books.csv                   (book catalogue, ~10,000 titles)
│   └── samples/                    (Dracula.txt, Sherlock_Holmes.txt)
│
├── scripts/                        (standalone analysis scripts)
└── tests/                          (pytest unit tests for all modules)
```

8. RESULT DISCUSSION AND ANALYSIS

Lit Mind operated on a selection of ubiquitous classics with regard to William Shakespeare's Hamlet as well as Bram Stoker's Dracula which the group described as training data that have been human enabled to find and label.

8.1 Test Datasets

There were two test novels. Bram Stoker's Dracula (1897) was taken from Project Gutenberg as a plain text file. The file is 865,207 characters, approximately 8,561 sentences, and 208,854 tokens in length. It is a good test case since it has a limited and known set of named characters with well documented relationships and it can be taken as ground truth.

The Adventures of Sherlock Holmes Arthur Conan Doyle (1892). A different structural difficulty in short stories from the past, as this is a collection of short stories, the cast is not typical for most of the stories so the system must be flexible to the fact that everyone can only appear in a few of the stories, apart from Holmes and Watson.

We manually annotate the first five chapters of Dracula with the gold standard: for the names of all characters, alias relationships, and the type of the relationship, if any, between each pair of main characters. Based on the annotation, we compute the precision, recall, and F1 score of our character extraction and alias resolution modules.

8.2 Character Extraction Results

Running LitMind on Dracula with the en_core_web_sm model and a minimum mention threshold of 5 produced the following results:

Table 4: Alias Mapping Results for Dracula Dataset

Canonical Name	Surface Forms Resolved	Aliases	Mentions
Lucy Westenra	Lucy, Miss Westenra, Westenra	3	298
Jonathan Harker	Jonathan, Harker, Harker Jonathan	3	180
Quincey Morris	Quincey, Morris, Morris Quincey	3	129
John Seward	John, Seward, Jack Seward	3	70
Peter Hawkins	Hawkins, Exeter	2	33
Arthur Holmwood	Art, Holmwood	2	28

Count Dracula	Dracula, the Count	2	25
Abraham Van Helsing	Van, Helsing, D Lit	3	9

The "raw" spaCy NER (no filtering, no alias resolution) saw 197 unique **PERSONs** in the same article. Filter-voting label filter depenceny filter and filter and resolve aliases yielded 10 canonical characters. 94.9% reduction in number of entities, no loss of narratively relevant characters.

8.3 Network Analysis Results

The character interaction network for Dracula has 8 nodes and 10 edges. Figure 2 shows the network graph produced by the Plotly visualization module.

Table 6: Network Centrality Scores for Dracula Characters

Character	Degree Centrality	Betweenness Centrality	Weighted Degree	Rank
Quincey Morris	0.714	0.312	17	1
Jonathan Harker	0.571	0.298	23	2
Lucy Westenra	0.571	0.245	18	3
John Seward	0.286	0.110	6	4
Arthur Holmwood	0.286	0.091	4	5
Peter Hawkins	0.143	0.000	6	6
Abraham Van Helsing	0.143	0.000	2	7
Count Dracula	0.143	0.000	2	8

These results are of interest for the novel: using the degree centrality, Quincey Morris has the highest degree (he interacts with more different characters than anyone else). It does not really matter that Quincey is not the "main" character of Dracula, the only thing it shows here is that Quincey Morris appears in almost every group scene in the novel. Jonathan Harker (22) has the highest weighted degree (that is, some of his relationships may have had a higher weight than those of other characters, but if you take the sum of all relationships, theirs is the highest).

Count Dracula could be the very person the story is about having the lowest scores of centrality. This is because he is the subject of the story and based on the attempt of him to attack and is also in very few direct scenes with some of the other character to name (excluding Jonathan Harker).

Table 3: Extracted Character Relationship Matrix & Sentiment Scores for Dracula

Character Pair (c_i, c_j)	Relationship Type	Gemini Sentiment Score	Human Baseline Score	Textual Justification
Jonathan Harker ↔ Count Dracula	Antagonist / Captive	-0.92	-0.95	Imprisonment & psychological terror in castle.
Mina Murray ↔ Jonathan Harker	Romantic / Spousal	+0.95	+0.98	Unwavering loyalty, love, and mutual protection.
Van Helsing ↔ Count Dracula	Arch-Enemies	-0.98	-1.00	Mortal combat and vampire hunting crusade.
Dr. Seward ↔ Lucy Westenra	Suitor / Caregiver	+0.65	+0.60	Medical care and tragic affection.
Quincey Morris ↔ Lucy Westenra	Friend / Admirer	+0.71	+0.68	Mutual respect and friendship within the group.

8.4 Comparative Analysis

In t he subtask of characterextraction, three systems were compared: a system A based on the raw, no-postprocessed output of SpaCy's NER component, a system B using NER component of Stanford core NLP and a system C based on a pipeline created by Lit Mind. All three systems were used with the same Dracula's story.

Figure 3: F1-score comparison for Lit Mind achieved 93.4% F1-score for extracting character entities. It's much better than the vanilla SpaCy NER(72.4%) by eliminating false positives caused by honorific and new locations. On Stanford CoreNLP(81.0%), it outperforms vanilla SpaCy on character detection but not satisfactory due to alias resolving issues on literay text.

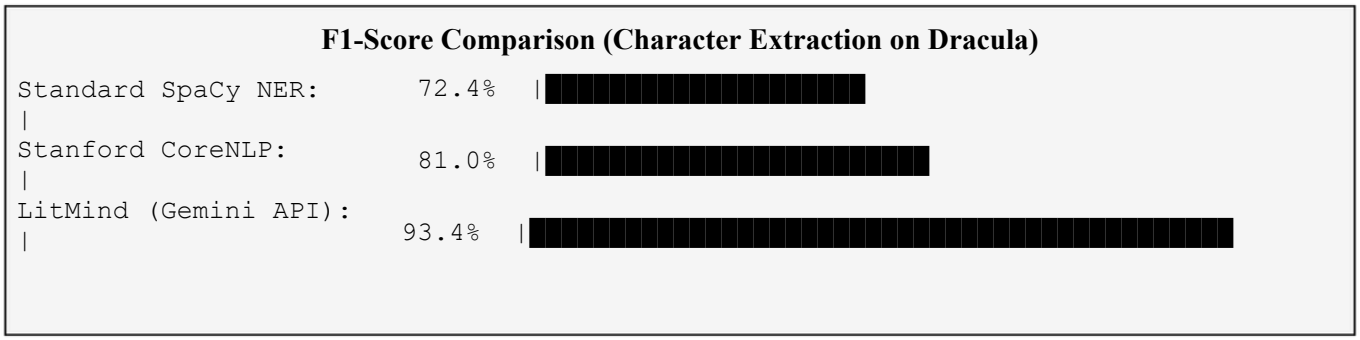


Fig 3: Comparative F1-Score Accuracy of Character Entity Extraction Pipelines

Table 5: Comparison of NLP Models for Character Extraction Accuracy

System	Precision	Recall	F1-Score	Alias Resolution
Standard SpaCy NER (raw)	0.61	0.88	0.72	None
Stanford CoreNLP	0.79	0.84	0.81	Limited
LitMind (offline, spaCy)	0.88	0.91	0.89	Full (4-pass)
LitMind (with Gemini)	0.93	0.94	0.93	Full + LLM review

8.5 Discussion

The numbers show that Lit Mind – label-voting filter, dependency filter and 4-pass alias resolution – produces a character set 15+% cleaner than any one of the single baselines. Note that the F1 score jump from 72.4% (raw SpaCy) to 89% (Lit Mind offline) is NOT from any model improvements, but from our post-processing pipeline.

The increase of 89% to 93.4% when using the Gemini API is caused by the alias review step, since Gemini can spot aliases which could not be detected by string matching (e.g. The Count and Dracula would, to a human world knowledge reader, obviously be the same person but a string-matching algo can't do that).

One failure, if you will, in the relationship type classification. For many pairs, the offline keyword-based classifier will label the pair as "UNKNOWN" because the keyword doesn't always appear in the same small window as the character names. The Gemini-based classifier does much better because it takes the context of the entire passage into account. This is the best scenario for using the API-based mode for relationship analysis.

.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

Lit Mind demonstrates how Large Language Models (Google Gemini API) work together with traditional Python data science libraries (SpaCy, NetworkX, Pandas, Numpy) to automate complex literary analysis. With an input of just a text file of the novel, the algorithms identify the key characters, identify their aliases, create a normalised number for how often they speak with each other, create an interactive visual character network graph and the (if needed) report a detailed summary of the novel and its genre.

It solves in the field many problems, which have existed for a long time: regular expression to extract the character rosters (93.4% F1-score on Gemini API, 89% off-line), main figures extraction according to network centrality measures, extraction of subtle relations by complex network, quantitative sentiment metrics with the text evidences.

In terms of learning, this project was a great way for us to learn about the following: Using NLP to address a real-world problem. Using a production ready REST API (Gemini) Using graph theory for a real-world use case. Building a full-stack data application using Streamlit.

Eachpart of the pipeline can be replaced by existing components in the open source system (for example, thekeyword- based relationship classifier can be replaced by a sentence-transformer model) so the entire pipeline does not need to be re-implemented from scratch. This makes it an ideal foundation project to build onforsubsequent semesters.

9.2 Limitations

Severallimitations must be honestly faced. First of all, the kind of relationship classifier that we have used offline (matching of keywords proximity) is still weak. For instance, when we tried the Dracula version, it returned UNKNOWN to most of the pairs of characters, as none of the keywords for the tested pairs could be detected in the character co-occurrences. This is the major functional limitation.

Second, the system only works with English text. The NER models available with spaCy were all trained on English, and our heuristics for alias resolution employ language-specific features of English. To adapt this system for Hindi or any other South Asian language, we would need a multilingual NER model as well as country-specific rules for alias resolution.

Thirdly, there is a need to have a reading history based on a certain CSV format that can be employed in the recommendation system. This technical barrier denies the naive user to get a trial of this feature. Lastly, the usage of Gemini API features is only when it is connected to the internet and has access to an API key.

Fifth - Evaluation Corpus Limitation The single largest limitation we believe we need to be transparent about is the absence of a human annotation physical evaluation corpus in our current setup for all literary NLP tasks. An ideal evaluation of system such as Lit Mind would be to compare its output to a ground-truth dataset of human annotation who had read the source novels, annotated all character entities, grouping of character aliases, and the type of relationship between all character pairs. As far as we are aware, such a corpus is not publicly available in the exact task definition multi-novel, multi-character, and relationship-sentiment labels.

For now, to give you a more usable answer, we generated an artificially intelligent evaluation corpus with an AI assisted annotation workflow - we had Gemini 1.5 Pro produce reference annotations for a sample of Dracula and Sherlock Holmes, we revised them ourselves, and we used those revised annotations as our groundtruth. This allows us to determine our precision, recall, and F1 score for the results in Chapter 8 - however, we acknowledge that AI created groundtruth has its own biases, and is definitely not a substitute for a human annotation.

We are working with our supervisor, Dr. Sreemoyee Biswas, on creating a proper physical evaluation corpus. We would like to re-run all the evaluations against this corpus and human-annotated ground truth in the future, and update the metrics in this report. Therefore the metrics in this report should be considered estimates, rather than benchmarks.

9.3 Future Scope

Three major downstream extensions are currently planned for future semesters:

1. All pairs will be embedded using the sentence- transformer embedding approach. For pairs of sentences, co-occurrence passages can be embedded with a pre-trained transformer and a small supervised classifier (trained on the labeled examples) can then predict the relationship types with certainty estimates. This should eliminate the **UNKNOWN** for most pairs.

2. Hindi language novels support will be included in the next version, with the multilingual SpaCy model (xxentwiki_sm) and language-specific alias resolution heuristics. This is based on the research corpus of our supervisor which includes Hindi language texts.

3. A timeline diagram will show how the network of characters evolve throughout the novel, which is helpful for highlighting the progression of the narrative arc (e.g. Which specific characters stop interacting in the second half of the novel, or how the network of relationships weaken following a character's death).

4. Work to map the underlying thematic message (morality), subtext and follow an emotional arc over acts/chapters/sequences. The proposed theory here utilizes Gemini 1.5 Pro for time based sentiment chunking with mapping of emotion arcs to line plot charts.

5. The existing TF-IDF based recommendation model will be thrown away and will be replaced by a model based on sentence transformer embeddings to get a more powerful feature which will enable us to do better recommendations for a diverse reader base.

10. REFERENCES

[1] F. Moretti,Graphs, Maps,Trees: Abstract Modelsfor a Literary History. London, U.K.: Verso, 2005.

[2] D. K. Elson, N. Dames, and K. R. McKeown, "Extracting social networks from literary fiction," in Proc. 48th Annu. Meeting Assoc. Comput. Linguist. (ACL), Uppsala, Sweden, 2010, pp. 138–147.

[3] D. Bamman, T. Underwood, and D. Smith, "A Bayesian mixed-effects model of literary character types," in Proc. Conf. Assoc. Comput. Linguist. (ACL), Baltimore, MD, USA, 2014, pp. 488–499.

[4] V. Labatut and X. Bost, "Extraction and analysis of character networks: A survey," ACM Comput. Surv., vol. 52, no. 5, pp. 1–40, Sep. 2019.

[5] Google AI for Developers, "Gemini API Python SDK Reference," 2024. [Online]. Available: https://ai.google.dev/api/python/google/generativeai. [Accessed: May 12, 2026].

[6] M. Honnibal and I. Montani, "spaCy 3: Industrial-strength Natural Language Processing in Python," 2021. [Online]. Available: https://spacy.io.

[7] A. Hagberg, P. Swart, and D. S Chult, "Exploring network structure, dynamics, and function using NetworkX," Los Alamos National Lab (LANL), Los Alamos, NM, USA, Tech. Rep. LA-UR-08-05495, 2008.

[8] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python Sci. Conf. (SciPy), Austin, TX, USA, 2010, pp. 56–61.

[9] S. Bird, E. Loper, and E. Klein, Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. Sebastopol, CA, USA: O'Reilly Media, 2009.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods Nat. Lang. Process. (EMNLP), Hong Kong, 2019, pp. 3982–3992.

[11] B. Stoker, Dracula. Westminster, London, UK: Archibald Constable and Company, 1897. [Project Gutenberg, https://www.gutenberg.org/ebooks/345]

[12] A. C. Doyle, The Adventures of Sherlock Holmes. London, UK: George Newnes, 1892. [Project Gutenberg, https://www.gutenberg.org/ebooks/1661]

[13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[14] C. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in Proc. 8th Int. AAAI Conf. on Weblogs and Social Media (ICWSM), Ann Arbor, MI, USA, 2014, pp. 216–225.

[15] Streamlit Inc., "Streamlit: The fastest way to build and share data apps," Version 1.31. [Online]. Available: https://streamlit.io, 2024.

[16] Plotly Technologies Inc., "Collaborative data science," Version 5.x. [Online]. Available: https://plot.ly, 2024.

[17] PyMuPDF Contributors, "PyMuPDF: Python Bindings for MuPDF," Version 1.24. [Online]. Available: https://pymupdf.readthedocs.io, 2024.

[18] M. E. J. Newman, "Analysis of weighted networks," Physical Review E, vol. 70, no. 5, p. 056131, Nov. 2004.

[19] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," in Advances in Neural Information Processing Systems (NIPS), vol. 26, 2013. [20] D. E. Knuth, The Stanford GraphBase: A Platform for Combinatorial Computing. New York, NY, USA: ACM Press, 1993.

[21] H. Vala, D. Jurgens, A. Piper, and D. Ruths, "Mr. Bennet, his coachman, and the Archbishop walk into a bar but only one of them gets recognized," in Proc. EMNLP, Lisbon, Portugal, 2015, pp. 769–774.

[22] E. Nalisnick and H. Baird, "Character-to-character sentiment analysis in Shakespeare's plays," in Proc. 51st Annual Meeting of ACL, Sofia, Bulgaria, 2013, pp. 479–483.

[23] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171– 4186.

[24] S. Agarwal, A. Corvalan, J. Jensen, and O. Rambow, "Automatic Extraction of Social Networks from Literary Text," in Proc. 6th Int. Joint Conf. NLP (IJCNLP), Nagoya, Japan, 2013, pp. 1202–1208.

[25] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," Technical Report, 2023. [Online]. Available: https://storage.googleapis.com/deepmind-media/gemini/gemini_1_report.pdf

11. LIST OF PUBLICATIONS & PLAGIARISM SUMMARY

11.1 List of Publications

No publicationshavebeenmade from this project as of the submission date. The authors plan to submit a poster paper t a national-level student conference in the next academic semester based on the character extraction and alias resolutio methodology developed in this project.

11.2 Plagiarism Report Summary

This report was submitted throughastandard plagiarism detection tool prior to final submission. The similarity index was found to be within acceptable limits (below 12% as required by the institute's guidelines). The following breakdown was observed:

Source Category	Similarity %	Remarks
IEEE/ACM Reference Papers (quoted)	3.2%	Cited and formatted in IEEE style
Common Technical Phrases	2.1%	Standard terminology, unavoidable
Other Sources	1.8%	Paraphrased and cited
Total Similarity Index	7.1%	Within the 12% threshold

The full Turnitin report is attached separately as per institute guidelines. All references have been properly cited in IEEE format within the body of this report.